

Volume Viii Numerical Foundations Reference

Contents

0.1	IEEE 754 Floating-Point Formats	3
0.2	Interval Arithmetic	4
0.2.1	Interval Arithmetic	4
0.2.2	Interval Arithmetic in $\mathbb{R}^n$	5
0.2.3	Week 1: Points, Vectors, and Floating-Point Geometry	5
0.2.4	Week 2: Linear Interpolation and Affine Sampling	5
0.3	Numerical Geometry	5
0.3.1	Projects	5
0.3.2	Exercises	5
0.4	Numerical PDE	6

Numerical Analysis

0.1 IEEE 754 Floating-Point Formats

IEEE 754 Floating-Point Formats

Definition 0.1.1 (IEEE 754 Binary Floating-Point Datum). An *IEEE 754 binary floating-point datum* is a binary encoding partitioned into three fields:

- a *sign field* consisting of one bit,
- an *exponent field* consisting of a fixed number of bits,
- a *fraction field* consisting of a fixed number of bits.

Its interpretation depends on the exponent-field pattern and the fraction-field pattern.

Definition 0.1.2 (Normalized IEEE 754 Binary Number). Let an IEEE 754 binary floating-point format have exponent bias b . A finite encoded datum is called *normalized* if its exponent field is neither all zeros nor all ones.

If s is the sign bit, if e is the unsigned integer determined by the exponent field, and if f is the binary fraction determined by the fraction field, then the represented value is

$$(-1)^s (1.f)_2 2^{e-b}.$$

Definition 0.1.3 (Subnormal IEEE 754 Binary Number). Let an IEEE 754 binary floating-point format have exponent bias b . A finite encoded datum is called *subnormal* if its exponent field is all zeros and its fraction field is not all zeros.

If s is the sign bit and if f is the binary fraction determined by the fraction field, then the represented value is

$$(-1)^s (0.f)_2 2^{1-b}.$$

Definition 0.1.4 (IEEE 754 Infinity Encoding). An IEEE 754 binary floating-point datum encodes *infinity* if its exponent field is all ones and its fraction field is all zeros.

The sign bit determines whether the represented value is $+\infty$ or $-\infty$.

Definition 0.1.5 (IEEE 754 NaN Encoding). An IEEE 754 binary floating-point datum encodes *NaN* (Not a Number) if its exponent field is all ones and its fraction field is not all zeros.

Definition 0.1.6 (IEEE 754 Binary16 Format). The *IEEE 754 binary16 format* is the 16-bit binary floating-point interchange format with the following field layout:

- 1 sign bit,
- 5 exponent bits,
- 10 fraction bits,

- exponent bias 15.

For a normalized finite binary16 datum, the represented value is

$$(-1)^s(1.f)_2 2^{e-15}.$$

Definition 0.1.7 (IEEE 754 Binary32 Format). The *IEEE 754 binary32 format* is the 32-bit binary floating-point interchange format with the following field layout:

- 1 sign bit,
- 8 exponent bits,
- 23 fraction bits,
- exponent bias 127.

For a normalized finite binary32 datum, the represented value is

$$(-1)^s(1.f)_2 2^{e-127}.$$

Definition 0.1.8 (IEEE 754 Binary64 Format). The *IEEE 754 binary64 format* is the 64-bit binary floating-point interchange format with the following field layout:

- 1 sign bit,
- 11 exponent bits,
- 52 fraction bits,
- exponent bias 1023.

For a normalized finite binary64 datum, the represented value is

$$(-1)^s(1.f)_2 2^{e-1023}.$$

Definition 0.1.9 (IEEE 754 Binary128 Format). The *IEEE 754 binary128 format* is the 128-bit binary floating-point interchange format with the following field layout:

- 1 sign bit,
- 15 exponent bits,
- 112 fraction bits,
- exponent bias 16383.

For a normalized finite binary128 datum, the represented value is

$$(-1)^s(1.f)_2 2^{e-16383}.$$

0.2 Interval Arithmetic

0.2.1 Interval Arithmetic

Definition 0.2.1 (Interval Addition). For closed intervals $[a, b]$ and $[c, d]$, their *sum* is

$$[a, b] + [c, d] = [a + c, b + d].$$

Definition 0.2.2 (Interval Subtraction). For closed intervals $[a, b]$ and $[c, d]$, their *difference* is

$$[a, b] - [c, d] = [a - d, b - c].$$

Definition 0.2.3 (Interval Multiplication). For closed intervals $[a, b]$ and $[c, d]$, their *product* is

$$[a, b] \cdot [c, d] = [\min(ac, ad, bc, bd), \max(ac, ad, bc, bd)].$$

Definition 0.2.4 (Interval Containment). $[a, b] \subseteq [c, d]$ if and only if $c \leq a$ and $b \leq d$.

0.2.2 Interval Arithmetic in $\mathbb{R}^n$

Definition 0.2.5 (Box / Product Interval). Let $a, b \in \mathbb{R}^n$ with $a \leq b$ coordinatewise (i.e., $a_i \leq b_i$ for all i). The *box* (or *interval vector*) determined by (a, b) is

$$[a, b] := \{x \in \mathbb{R}^n : a \leq x \leq b\} = \prod_{i=1}^n [a_i, b_i].$$

Definition 0.2.6 (Box Containment). For $a \leq b$ and $c \leq d$ in $\mathbb{R}^n$, we write $[a, b] \subseteq [c, d]$ iff

$$(\forall x \in \mathbb{R}^n) (x \in [a, b] \Rightarrow x \in [c, d]).$$

Equivalently (and most useful computationally),

$$[a, b] \subseteq [c, d] \iff c \leq a \wedge b \leq d \quad (\text{coordinatewise}).$$

Definition 0.2.7 (Minkowski Addition of Boxes). For boxes $X = [a, b]$ and $Y = [c, d]$ in $\mathbb{R}^n$, their *Minkowski sum* is

$$X \oplus Y := \{x + y : x \in X, y \in Y\}.$$

For boxes, this closes in the class of boxes and satisfies the endpoint rule

$$[a, b] \oplus [c, d] = [a + c, b + d].$$

Definition 0.2.8 (Box Subtraction / Difference). For boxes $X = [a, b]$ and $Y = [c, d]$ in $\mathbb{R}^n$, define

$$X \ominus Y := \{x - y : x \in X, y \in Y\}.$$

Then

$$[a, b] \ominus [c, d] = [a - d, b - c].$$

Definition 0.2.9 (Scalar Multiplication of a Box). For $\lambda \in \mathbb{R}$ and a box $[a, b] \subseteq \mathbb{R}^n$, define

$$\lambda[a, b] := \{\lambda x : x \in [a, b]\}.$$

Then

$$\lambda[a, b] = \begin{cases} [\lambda a, \lambda b], & \lambda \geq 0, \\ [\lambda b, \lambda a], & \lambda < 0, \end{cases}$$

where the inequalities are coordinatewise.

Definition 0.2.10 (Coordinatewise / Hadamard Product of Boxes). For $x, y \in \mathbb{R}^n$, write $(x \odot y)_i := x_i y_i$. For boxes $X = [a, b]$ and $Y = [c, d]$, define

$$X \odot Y := \{x \odot y : x \in X, y \in Y\}.$$

Then $X \odot Y$ is again a box, computed coordinatewise:

$$[a, b] \odot [c, d] = \prod_{i=1}^n [\min S_i, \max S_i], \quad S_i := \{a_i c_i, a_i d_i, b_i c_i, b_i d_i\}.$$

0.2.3 Week 1: Points, Vectors, and Floating-Point Geometry

0.2.4 Week 2: Linear Interpolation and Affine Sampling

0.3 Numerical Geometry

0.3.1 Projects

0.3.2 Exercises

Proofs

Numerical PDE

0.4 Numerical PDE

Proofs